Team: **Dark_Knights**
Ratnala Aneesh • Raghoothama T R • Vatsal Soni

Financial Parallelverse

Complete Project Architecture, Engineering & Mathematical Deep-Dive

Product Vision: Explore your financial futures before making a decision — Deterministic Multi-Timeline Simulation Engine.

---

1. Executive Summary & Problem Solved

The FinTech Problem: Most young professionals, college students, and first-time loan applicants evaluate affordability solely on whether the monthly EMI fits within their current monthly salary. This creates a critical blind spot known in finance as *tail-risk vulnerability*. If a person commits to an EMI of ■5,686/month for a ■70,000 laptop, standard calculators confirm they have sufficient salary today. But what happens if their company delays salaries by 30 days? What if a medical emergency demands ■25,000 in Month 3? What if inflation increases daily living expenses by 15%? Conventional calculators cannot answer these life-altering questions.

The Innovation (Financial Parallelverse): Our application takes a user's real or hypothetical financial situation and simultaneously projects **5 concurrent deterministic futures** over a 6 to 24 month timeline. It introduces **Financial Breaking-Point Detection** (which identifies the exact month when savings turn negative or dip below emergency thresholds), evaluates **4 distinct purchase pathways**, and generates **downloadable iCalendar (.ics) reminders** for scheduled installments.

Layer	Technology	Key Responsibilities
Backend API	Python 3.13 / FastAPI / Uvicorn	REST API endpoints, Pydantic request validation, CORS, OpenAPI docs.
Financial Engine	Deterministic Python Algorithms	Reducing-balance EMI formula, free cash flow, breaking point detection.
Primary UI	Streamlit + Plotly + Custom CSS	High-end Midnight/Emerald SaaS theme, interactive multi-line charts.
Alternative UI	React 18 + TypeScript + Vite	Full enterprise client code in frontend_react/ connecting to REST API.
Database	SQLite + SQLAlchemy 2.x	Persistent profiles, active loan plans, and month-by-month payment tracking.
Calendar Sync	Python RFC 5545 iCalendar (.ics)	Exportable calendar files with VALARM notifications for phones/Google Cal.

2. Mathematical Formulations & Financial Algorithms

Unlike AI wrappers that hallucinate numbers, all calculations in Financial Parallelverse are **100% deterministic**, mathematically verified, and reproducible:

A. Reducing-Balance EMI Formula

For a financed principal P (Purchase Price – Down Payment), monthly interest rate $r = (Annual\ Rate / 12) / 100$, and tenure of n months:
$$\mathbf{EMI = P \times r \times (1 + r)^n / ((1 + r)^n - 1)}$$
When $r = 0$ (Zero-Cost EMI): $\mathbf{EMI = P / n}$.
In each month m: Interest Portion = Balance × r; Principal Paid = EMI – Interest Portion. Final month balance reconciles to exactly ■0.00.

B. Cash Flow Dynamics & Runway Metrics

- **Monthly Free Cash Flow:** Income – (Essential Expenses + Discretionary Expenses + Existing Debt EMIs)
- **Savings Rate %:** (Free Cash Flow / Monthly Income) × 100
- **Emergency Runway (Months):** Liquid Savings / (Essential Expenses + Existing Debt EMIs)
- **Debt-to-Income (DTI) %:** (Existing Loan EMIs / Monthly Income) × 100 (Safe threshold: < 35%)

C. Month-by-Month Deterministic Simulation Order

1. Opening Savings(m) = Closing Savings(m – 1) [Month 1 Opening = Current Savings]
2. Income Received(m): Set to ■0 if salary is delayed; double income arrives upon catch-up.
3. Routine Expenses(m): (Essential + Discretionary) × Inflation Multiplier.
4. Debt & Purchase Outflows(m): Existing EMI + New Purchase Payment (Down payment in M1, EMI thereafter).
5. Emergency Shock(m): Charged exclusively in the specified shock month (e.g. Month 3).
6. Closing Savings(m) = Opening + Inflows – All Outflows.
7. Breaking Point: Triggered immediately if Closing Savings < 0 (Deficit) or < Emergency Target.

3. Deep-Dive: The 5 Simulated Parallel Timelines

The flagship feature of Financial Parallelverse is simulating five concurrent realities to evaluate resilience:

• Timeline 1: Baseline Reality (The Control) — *Green (#10b981)*

Assumes steady income received on time, routine expenses, and no unforeseen financial shocks. Demonstrates normal capital accumulation.

• Timeline 2: Emergency Medical / Appliance Shock — *Amber (#f59e0b)*

Simulates a sudden out-of-pocket expense (default ■25,000) in Month 3. Tests whether liquid reserves absorb the shock without breaching the emergency target.

• Timeline 3: Delayed Salary Timing Shift — *Cyan (#06b6d4)*

Simulates a delayed paycheck in Month 2 where income received is ■0. Living costs and loan EMIs must still be paid out of reserves. Double pay arrives in Month 3 upon resolution.

• Timeline 4: Cost of Living Inflation (+15%) — *Violet (#a855f7)*

Applies a permanent 15% increase to all essential and discretionary expenses, reflecting sudden inflation, rent hikes, or utility price increases.

• Timeline 5: Parallel Nightmare (Combined Adversity) — *Crimson (#f43f5e)*

The ultimate stress test: Salary delayed in Month 2, sudden ■25,000 emergency in Month 3, and 15% inflation running simultaneously. Exposes catastrophic breaking points.

4. Purchase Affordability Decision Matrix (■70,000 Laptop Example)

When evaluating a major purchase (e.g. ■70,000 laptop), the platform presents 4 clear pathways with automated trade-off scoring:

Option	Upfront Cash	Monthly Drag	Safety Assessment & Trade-off
1. Buy Now with Cash	■70,000	■0/mo	Zero interest fees, but instantly depletes liquid emergency reserve by 58%.
2. Save & Buy Later	■0 today	■0/mo	Protects liquid reserve 100%. Purchases using surplus savings in 4 months.
3. Cheaper Alternative	■42,000	■0/mo	Satisfies 90% of functional requirements while keeping reserves above target.
4. Finance via EMI	■10,000 (Down)	■5,330/mo	Preserves liquid cushion; costs ■3,971 in interest over 12 months.

5. Complete Project File Directory & Components

All code was created and committed into a clean, modular structure inside C:\Users\Raghoothama T R\Desktop\Financial_Parallelverse:

- **backend/app/services/financial_engine.py**: Core baseline formulas: Free cash flow, savings rate, runway months, and DTI ratio.
- **backend/app/services/scenario_engine.py**: Flagship multi-timeline simulation engine projecting all 5 futures with breaking point alerts.
- **backend/app/services/affordability.py**: 4-way purchase decision evaluator (Cash vs Save vs Cheaper vs EMI).
- **backend/app/services/emi_calculator.py**: Reducing-balance amortization calculator supporting zero interest and extra repayments.
- **backend/app/services/reminder_service.py**: Installment scheduler, month-end date clamping, and RFC 5545 .ics iCalendar generator.
- **backend/app/api/**: FastAPI REST routers: health.py, profile.py, scenarios.py, purchases.py, emi.py, installments.py.
- **backend/app/main.py**: FastAPI app factory, CORS middleware, and automatic database initialization.
- **backend/tests/**: Comprehensive Pytest suite with 23 passing tests covering all calculations and API routes.
- **frontend_streamlit/app.py**: Ultra-polished Streamlit UI with Plotly charts, dark obsidian theme, persona presets, and .ics export.
- **frontend_react/**: Full React 18, TypeScript, and Vite source tree with modular page components.

- **scripts/seed_demo.py**: Seeds database with realistic Indian Rupee data (Alex Johnson profile & 2 active EMI plans).
- **run_all.bat**: 1-Click launcher starting both FastAPI Backend (port 8000) and Streamlit UI (port 8501).

6. FastAPI Backend REST API Contract

Method	Endpoint Path	Request / Response Role
GET	/api/health	System status, service version, and server timestamp.
GET / PUT	/api/profile	Retrieve and update user financial profile in SQLite.
POST	/api/scenarios/simulate	Execute 5-timeline simulation with custom shock inputs.
POST	/api/purchases/evaluate	Compare 4 purchase pathways with risk scoring.
POST	/api/emi/calculate	Calculate loan EMI, interest totals, and full amortization schedule.
GET / POST	/api/installments	List all active commitments or create a new loan plan.
PATCH	/api/installments/{id}/payments/{pid}	Toggle payment status between PAID and PENDING.
GET	/api/installments/{id}/calendar	Download RFC 5545 .ics iCalendar file for calendar sync.

7. Step-by-Step Execution Guide

To run the complete system on Windows:

Step 1: Automated Test Suite

```
cd "C:\Users\Raghoothama T R\Desktop\Financial_Parallelverse"
.\run_tests.bat
# Or: python -m pytest backend/tests -v
```

Step 2: Seed Demo Data

```
python scripts/seed_demo.py
```

Step 3: Launch Both Servers (1-Click)

```
.\run_all.bat
# Opens FastAPI Backend (:8000) and Streamlit FinTech UI (:8501)
```

8. Live Hackathon Pitch & Judge Demonstration Script

Deliver this exact 3-minute pitch to the judges during your live demonstration:

Minute 1 (The Hook): 'Good evening judges! We are Team Dark_Knights presenting Financial Parallelverse for The Vampire's Ledger track. Today, millions make big financial decisions—like buying a ■70,000 laptop—using standard EMI calculators that only tell them if their current paycheck can make month 1's installment. But life isn't static. A delayed salary or a medical emergency can instantly drain their reserves.'

Minute 2 (The Simulator Demo): 'Watch what happens when we input a ■70,000 laptop into our Parallel Futures Engine. We simulate 5 simultaneous realities. In Baseline, savings grow steadily. But in Timeline 2, an unexpected ■25,000 hospital bill breaches our emergency fund. In Timeline 5—the combined nightmare—our engine flags an exact breaking point in Month 3 where cash drops negative. No standard tool does this!'

Minute 3 (Action & Reminders): 'Next, our Purchase Matrix compares 4 options: cash, waiting, a cheaper ■42k model, or structured EMI. Finally, in our Installment Tracker, users can download an official .ics iCalendar file with one click to sync due-date alarms right to their phone. Everything is backed by FastAPI, SQLite, and 23 verified mathematical tests. Thank you!'